

8.1

Question

List three examples of deadlocks that are not related to a computer system environment.

Answer

1. **Traffic intersection deadlock**
Cars enter an intersection from four directions and each waits for another car to move.
 2. **Two people in a narrow hallway**
Each person waits for the other to step aside, so neither moves.
 3. **Dining-table deadlock**
Several people each hold one utensil and wait for another utensil held by someone else.
-

8.2

Question

Suppose that a system is in an unsafe state. Show that it is possible for the threads to complete their execution without entering a deadlocked state.

Answer

An unsafe state does **not** mean deadlock has already occurred. It only means that deadlock **may** occur.

Example:

- Two threads: T0 and T1
- One resource type with 2 instances.

Current allocation:

Thread	Allocated	Max
T0	1	2
T1	1	2

Available = 0

This is unsafe because if both request another resource simultaneously, deadlock occurs.

But if:

1. T0 finishes without requesting more,
2. T0 releases its resource,
3. T1 then gets the released resource and finishes,

then no deadlock occurs.

So an unsafe state can still allow successful completion.

8.3

Question

Given:

Thread	Allocation (A B C D)	Max (A B C D)
T0	0 0 1 2	0 0 1 2
T1	1 0 0 0	1 7 5 0
T2	1 3 5 4	2 3 5 6
T3	0 6 3 2	0 6 5 2
T4	0 0 1 4	0 6 5 6

Available = (1 5 2 0)

Using Banker's Algorithm:

(a) Find Need matrix

(b) Is system safe?

(c) Can request by T1 for (0,4,2,0) be granted?

(a) Need Matrix

Need = Max – Allocation

Thread	Need (A B C D)
T0	0 0 0 0
T1	0 7 5 0
T2	1 0 0 2
T3	0 0 2 0
T4	0 6 4 2

(b) Safe State?

Available = (1 5 2 0)

Step 1: T0 can finish

Need = (0 0 0 0)

New Available:
 (1 5 2 0) + (0 0 1 2)
 = (1 5 3 2)

Step 2: T2 can finish

Need = (1 0 0 2)

Available becomes:
 (2 8 8 6)

Step 3: T3 finishes

Available:
 (2 14 11 8)

Step 4: T4 finishes

Available:
 (2 14 12 12)

Step 5: T1 finishes

Safe sequence:

T0 → T2 → T3 → T4 → T1

Hence system is **safe**.

(c) Request by T1 = (0,4,2,0)

Check:

1. Request $\leq$ Need
 $(0,4,2,0) \leq (0,7,5,0) \checkmark$
2. Request $\leq$ Available
 $(0,4,2,0) \leq (1,5,2,0) \checkmark$

Temporarily allocate:

Available becomes:

$(1,1,0,0)$

T0 can still finish first and system remains safe.

Therefore request **can be granted immediately**.

8.4

Question

Compare containment scheme with circular-wait prevention scheme.

Answer

Containment Scheme

A thread must first acquire a global lock F before requesting any other lock.

Advantages:

- Very simple.
- Completely prevents circular wait.

Disadvantages:

- Poor concurrency.
 - Only one thread can enter resource-acquisition region at a time.
-

Circular-Wait Scheme

Resources are numbered and must always be requested in increasing order.

Advantages:

- Better parallelism.
- Multiple threads can operate simultaneously.

Disadvantages:

- More difficult to implement.
 - Requires global ordering discipline.
-

8.5

Question

Prove safety algorithm requires order of mn^2 operations.

Answer

Where:

- (m) = number of resource types
- (n) = number of threads

The algorithm repeatedly searches all (n) threads to find one whose $\text{Need} \leq \text{Available}$.

For each thread comparison:

- Compare (m) resource types.

Worst case:

- Up to (n) passes.
- Each pass checks (n) threads.

- Each check compares (m) resources.

Total operations:

$$n \times n \times m = O(mn^2)$$

8.6

Question

Arguments for and against installing deadlock avoidance.

Answer

(a) Arguments FOR

- Prevents deadlocks entirely.
- Saves rerun costs.
- Improves reliability.
- Prevents wasted CPU work.

Current loss:

- 2 deadlocks/month
- 10 jobs terminated each
- Jobs half complete
- Cost:

$$2 \times 10 \times 2 \times \frac{1}{2} = \$20/month$$

(b) Arguments AGAINST

- 10% execution-time overhead on all jobs.
 - Turnaround increases by 20%.
 - System already tolerates deadlocks reasonably well.
 - Deadlocks are relatively rare.
-

8.7

Question

Can a system detect starvation?

Answer

Yes.

The system can monitor:

- Waiting time of threads.
- Number of times resources are denied.

If one thread waits excessively while others continue progressing, starvation is detected.

Solutions:

- Aging
 - Fair scheduling
 - FIFO resource allocation
-

8.8

Question

Analyze preemption-based resource policy.

(a) Can deadlock occur?

No.

Because resources can be taken away from blocked threads, the **no-preemption condition** is violated.

Since one necessary deadlock condition is absent, deadlock cannot occur.

(b) Can indefinite blocking occur?

Yes.

A blocked thread may repeatedly lose resources to newer requests and never progress.

Thus starvation/indefinite blocking is possible.

8.9

Question

Determine whether states are safe.

(Need = Max – Allocation)

Thread	Need
T0	2 1 0 3
T1	1 0 0 1
T2	0 2 0 0
T3	4 1 0 2
T4	2 1 1 3

(a) Available = (0,3,0,1)

Possible order:

T2 → T1 → T0 → T3 → T4

Therefore state is **safe**.

(b) Available = (1,0,0,2)

Only T1 can finish first.

After T1:
Available = (3,2,1,2)

Then others finish safely.

Safe state.

8.10

Question

Can safety algorithm code be reused for deadlock detection?

Answer

No.

Reason:

- Banker's algorithm assumes processes may still request up to Max.
- Deadlock detection uses current waiting requests only.

Setting:

$$Max_i = Waiting_i + Allocation_i$$

does not correctly represent future behavior and may produce incorrect conclusions.

8.11

Question

Can deadlock involve only one single-threaded process?

Answer

No.

Deadlock requires a cycle of waiting.

A single-threaded process cannot wait on itself while holding resources in the required circular dependency.

At least two execution entities are required.

8.12

Question

Consider traffic deadlock in Figure 8.11.

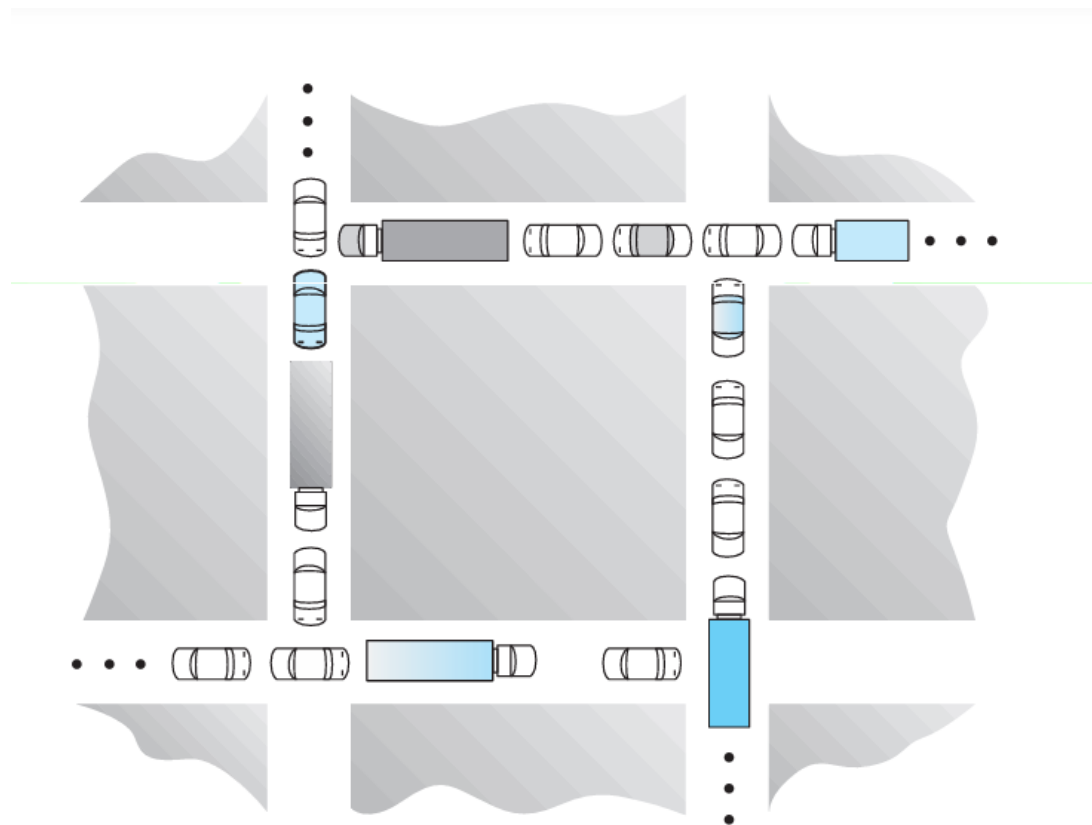


Figure 8.11 Traffic deadlock for Exercise 8.12.

(a) Show four necessary conditions hold.

Answer

1. Mutual Exclusion

Only one car can occupy a road section at a time.

2. Hold and Wait

Each car holds its current road segment while waiting for next segment.

3. No Preemption

Cars cannot forcibly remove other cars.

4. Circular Wait

Each car waits for another car in a cycle around the intersection.

Hence deadlock exists.

(b) Simple rule to avoid deadlock

A car may enter intersection only if it can completely pass through without stopping.

Equivalent:

“Do not enter unless exit space is available.”

8.13

Question

Draw resource-allocation graph for Figure 8.1 deadlock example.

Answer

Graph cycle:

Meaning:

- P1 holds R2 and waits for R1
- P2 holds R1 and waits for R2

This cycle represents deadlock.

8.14

Question

Draw graph for semaphore deadlock involving P0, P1 and semaphores S and Q.

Answer

Cycle:

[
P_0 $\rightarrow$ Q $\rightarrow$ P_1 $\rightarrow$ S $\rightarrow$ P_0
]

- P0 holds S and waits for Q.
- P1 holds Q and waits for S.

Deadlock occurs.

8.15

Question

Can deadlock occur with only reader-writer locks?

Answer

Yes.

If multiple reader-writer locks exist and threads acquire them in conflicting order, circular wait may occur.

Example:

- T1 holds lock A, waits for B
- T2 holds B, waits for A

Thus deadlock remains possible.

8.16

Question

Role of CPU scheduler in deadlock.

Answer

Deadlock depends on execution order.

Example:

- If scheduler allows T1 to acquire one lock,
- then switches to T2 which acquires another lock,
- both may later wait forever.

Different scheduling orders may avoid deadlock entirely.

Thus scheduler timing contributes to deadlock occurrence.

8.17

Question

Fix transaction() to prevent deadlock.

Answer

Always acquire locks in fixed order.

Example pseudocode:

```
if (lock1 < lock2) {  
    acquire(lock1);  
    acquire(lock2);  
}  
else {  
    acquire(lock2);  
    acquire(lock1);  
}
```

This prevents circular wait.

8.18

Question

Which graphs in Figure 8.12 show deadlock?

Answer

(a)

No deadlock.

Safe order:

$T_1 \rightarrow T_2 \rightarrow T_3$

(b)

Deadlock exists.

Cycle:

```
[  
T_1 \rightarrow R_1 \rightarrow T_3 \rightarrow R_3 \rightarrow T_2 \rightarrow R_2  
\rightarrow T_1  
]
```

(c)

No deadlock.

Possible order:
 $T_2 \rightarrow T_1 \rightarrow T_3$

(d)

Deadlock exists.

Cycle:

[
 $T_1 \rightarrow R_2 \rightarrow T_4 \rightarrow R_1 \rightarrow T_1$
]

Another cycle also exists involving T_2 .

(e)

No deadlock.

Possible order:
 $T_2 \rightarrow T_4 \rightarrow T_1 \rightarrow T_3$

(f)

No deadlock.

Possible order:
 $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4$

8.19

Question

Compare circular-wait prevention vs deadlock avoidance.

Answer

(a) Runtime overhead

Circular Wait

- Very low overhead.
- Only ordering checks needed.

Banker's Algorithm

- High overhead.
 - Safety checks on every request.
-

(b) System throughput

Circular Wait

- Better throughput.
- Less computation.

Banker's Algorithm

- Lower throughput.
 - Some safe requests delayed.
-

8.20

Question

Effects of changes under Banker's Algorithm.

Answer

(a) Increase Available

Safe always.

(b) Decrease Available

Safe only if resulting state remains safe.

(c) Increase Max

Safe only if system still safe after change.

(d) Decrease Max

Always safe.

(e) Increase number of threads

Safe only if resulting state is safe.

(f) Decrease number of threads

Always safe because resources are freed.

8.21

Question

Find Need matrix.

Need = Max – Allocation

Thread	Need
T0	4 2 2 1
T1	2 1 0 2
T2	4 3 0 2
T3	3 1 1 1
T4	4 2 3 1

8.22

Question

Show system with 4 resources and 3 threads needing at most 2 each is deadlock free.

Answer

Maximum total held before completion:

$$\begin{bmatrix} 3 \\ 3 \times (2-1) = 3 \end{bmatrix}$$

Since system has 4 resources, at least one thread can always obtain its second resource and complete.

After completion it releases resources.

Thus deadlock cannot occur.

8.23

Question

Show deadlock freedom conditions.

Answer

Given:

$$\left[\sum \text{Max}_i < m+n \right]$$

Assume deadlock occurs.

Then every thread must hold at least one resource and wait for another.

If each thread holds at most $(\text{Max}_i - 1)$ resources:

$$\left[\sum (\text{Max}_i - 1) < m \right]$$

Thus at least one resource remains free.

Therefore one thread can proceed.

Contradiction.

Hence deadlock impossible.

8.24

Question

Dining philosophers with any two chopsticks.

Answer

Grant request only if:

After granting, at least one philosopher can still obtain two chopsticks and eat completely.

Equivalently:

- Never allow all chopsticks to become partially allocated.

8.25

Question

Each philosopher now needs 3 chopsticks.

Answer

Safe rules:

1. Never allocate all chopsticks if nobody can complete.
 2. Grant request only if at least one philosopher can still obtain 3 chopsticks.
 3. Prevent circular partial allocations.
-

8.26

Question

Why single-resource Banker's separately per resource fails.

Answer

Example:

Two resources A and B.

Available:

(1,1)

Threads:

Thread Need	
T1	(1,0)
T2	(0,1)
T3	(1,1)

Per-resource analysis says both resource types are safe individually.

But globally:

- T1 may get A
- T2 may get B
- T3 blocked forever

Thus independent analysis fails.

8.27

Question

Determine safety.

Need matrix:

Thread	Need
T0	3 1 1 4
T1	2 3 1 2
T2	2 4 1 1
T3	1 4 2 2
T4	2 1 1 1

(a) Available = (2,2,2,3)

Safe sequence:

$T4 \rightarrow T1 \rightarrow T2 \rightarrow T3 \rightarrow T0$

Safe.

(b) Available = (4,4,1,1)

Safe sequence exists.

Safe state.

(c) Available = (3,0,1,4)

No thread can safely proceed initially.

Unsafe.

(d) Available = (1,5,2,2)

Safe sequence:

$T3 \rightarrow T1 \rightarrow T2 \rightarrow T4 \rightarrow T0$

Safe.

8.28

Question

Using Banker's Algorithm.

Need matrix:

Thread	Need
T0	3 3 3 2
T1	2 1 3 0
T2	0 1 2 0
T3	2 2 2 2
T4	3 4 5 4

Available = (2 2 2 4)

(a) Safe sequence

$T2 \rightarrow T1 \rightarrow T0 \rightarrow T3 \rightarrow T4$

System safe.

(b) T4 request (2,2,2,4)

After allocation system unsafe.

Cannot grant.

(c) T2 request (0,1,1,0)

Can grant safely.

(d) T3 request (2,2,1,2)

Cannot grant immediately because state becomes unsafe.

8.29

Question

Optimistic assumption in deadlock detection.

Answer

Assumption:

A waiting thread will eventually obtain requested resources.

Violation:

If resources are permanently held or cyclic waiting exists, assumption fails and deadlock occurs.

8.30

Question

Design bridge algorithm preventing deadlock.

Answer

```
mutex bridge;

northbound() {
    wait(bridge);
    cross_bridge();
    signal(bridge);
}

southbound() {
    wait(bridge);
    cross_bridge();
    signal(bridge);
}
```

Only one direction uses bridge at a time.

Deadlock prevented.

8.31

Question

Modify to prevent starvation.

Answer

Use:

- waiting counters
- turn variable
- condition variables/FIFO queue

Alternate access when both sides waiting.

This ensures fairness.

8.33

Question

Use `System.identityHashCode()` to prevent deadlock.

Answer

Acquire locks according to hash order:

```
Object first = obj1;
Object second = obj2;

if (System.identityHashCode(first) >
    System.identityHashCode(second)) {

    first = obj2;
    second = obj1;
}

synchronized(first) {
    synchronized(second) {
        // transaction
    }
}
```

Consistent ordering prevents circular wait.